

PENGWIN 2026 Task 1 — V1.3.3 STU-Net Two-Stage Submission

Algorithm Description

2026-06-07

V1.3.3 is an inference-only refinement on top of V1.3.2 (the model is **unchanged**). V1.3.2 took the leaderboard from ~0 to **fracture Dice 0.767** by fixing a random-weight bug (below); V1.3.3 then targets the dominant remaining error revealed by the official per-case metrics — **under-segmentation** (instance recall 0.456, merge_error 0.65, split_error 0). A multi-agent code audit with adversarial verification found the cause: the deploy decoder’s non-Sacrum **size-ratio merge** (0.05) was collapsing genuine 1–11 cm³ fracture fragments — and had silently **diverged** from the validated offline evaluator, which uses 0.0. Disabling it (Sacrum keeps its deliberate 0.10 precision guard) recovers the real fragments: on a local instance-level proxy, mean recall **0.46 → 0.51**, F1 **0.50 → 0.55**, with **split_error still 0 and precision held**. V1.3.3 also removes a speckle-era “bbox > 50 %” band-aid that fired on *clean* large anatomy in a tight FOV (swapping the soft probability ROI for a hard bone mask → out-of-distribution decode), and deletes ~120 lines of dead/legacy code. A confidence-based routing guard for the cross-group hallucination was prototyped and **rejected on the evidence** — Ds539 is *confidently wrong* (real and phantom bones both score 0.97–0.99 softmax), so that is a model-level issue for future retraining, not an inference threshold.

V1.3.2 fixes the *actual* root cause of the earlier ~0 Grand-Challenge scores and keeps the V1 STU-Net model and I/O contract unchanged. The GC container pins nnunetv2==2.5.1, whose `initialize_from_trained_model_folder()` builds the network but does **not** load the checkpoint into `predictor.network` (it defers the load to `perform_actual_prediction()`). Our custom inference path reads `predictor.network` directly and never triggers that deferred load, so **both stages ran with random initialization** → speckle anatomy (tens of thousands of connected components) → ~0. The local dev env happened to run nnUNet **2.5.2**, which *does* load at init (MIC-DKFZ/nnUNet#2520), so the bug was invisible locally. V1.3.2 loads the trained weights into the network **explicitly** in `build_predictor`, making inference correct on any nnUNet version. It also retains the V1.3 robust per-anatomy routing and the self-diagnostic logging (now including a per-stage weight checksum `w0sum`). **Model weights are identical to V1.**

0. Submission intent

Per-fragment instance segmentation of pelvic and femoral bone fractures (sacrum, left/right hip, femur) in CT. A **STU-Net two-stage anatomy-conditioned pipeline, femur fully modeled**, trained on a leakage-free, case-grouped split. Supersedes the V0.3.x pipeline-test (ResEnc, pelvic-only, femur-stub).

1. Method overview

A two-stage, anatomy-conditioned cascade wrapped in a robustness shell.

- **Stage 1 — anatomy** (Dataset539_PelvicFemurAnatomyV3): a whole-CT 5-class semantic model (0=bg, 1=sacrum, 2=leftHip, 3=rightHip, 4=femur).
- **Per-anatomy ROI**: for each present bone, take the bbox of the Stage-1 probability ≥ 0.5 (padded), and build a **3-channel ROI** — bone-LUT CT, Stage-1 anatomy probability, and a signed distance map clipped to ± 40 mm.
- **Stage 2 — fracture** (Dataset538_PelvicFemurBICMFragmentV5): a per-anatomy ABBC model that emits a 4-class field (background / border / boundary / core).
- **Decoder**: core-seed watershed — connected components of the core class are watershed seeds; support is flooded from them; a ≥ 1 cm³ component prune and a small-fragment merge follow.
- **Assembly**: per-bone fragment IDs are offset into the official PENGWIN ranges (sacrum 1-50, leftHip 51-100, rightHip 101-150, **femur 151-200**) and the volume is reoriented to the input frame.

Backbone. Both stages use **STU-Net-B** (58 M params, Apache-2.0) warm-started from a TotalSegmentator bone pretrain — a license-clean transfer that initializes the encoder for pelvic/femoral bone.

2. Stage 1 — Ds539 anatomy

- STU-Net-B, nnUNetResEncUNetLPlans / 3d_fullres, 1-channel CT input, 5-class softmax. Trainer PenguinTrainerSTUNetBaseAnatomyV301.
- Input: LPS canonicalize + raw HU fed through nnUNet’s own CTNormalization (foreground-percentile clip + z-score) and resampling to the model’s target spacing — the standard nnUNet preprocessing the model was trained with.
- Output: full-CT 5-class anatomy probability, resampled back to the CT grid.

3. Per-anatomy routing (robust — no pelvic/femur gate) [changed in V1.3]

Ds539’s marginal training hallucinates the cross-group anatomy: a pelvic case gets a phantom femur channel, and a femur case gets phantom pelvic channels. V1 gated the **whole case** to “pelvic” or “femur” by a single femur/pelvic argmax volume ratio (threshold 0.45); a borderline hallucination flipped the ratio and routed the case to the **wrong** anatomy set, scoring it **0**. End-to-end testing showed this misrouted ~25 % of cases (both directions).

V1.3 removes the gate: it processes **every anatomy whose Ds539 argmax mask is at least 20 % of the largest present mask**. The genuinely-present bone(s) are thus kept every time — a small hallucination is dropped by the fraction gate, a sizable one becomes a minor false-positive, never a zero. End-to-end batch (8 cases): **mean fracture Dice 0.726 → 0.968, zero 0-score cases**.

4. Stage 2 — Ds538 ABBC fracture

- STU-Net-B, 3-channel input, 4-class ABBC head. Trainer ...DenseCandidateCore025StrongPeakNoContactABBCSTUNetBV301 (a V300 boundary-attention refinement). Aligned with the PENGWIN 2024 1st-place (MIC-DKFZ) ABBC formulation: explicit contact-voxel classification is dropped in favour of a core-seed -> boundary watershed.
- Training: unified **case-grouped** 5-fold split (seed 12345) shared with Stage 1, so a held-out fold is held out across *both* stages (no patient leakage).

5. Core-seed watershed decoder

Core class -> `scipy.ndimage.label` seeds; watershed flood over the support mask; ≥ 1 cm³ connected-component prune; small-fragment merge. An anatomy-specific over-segmentation control merges the sacrum's spurious core islands (Sacrum Instance-F1 0.585 -> 0.892) while the multi-fragment hips/femur keep the defaults.

6. Robustness shell

L1) bone-skeleton anatomy decomposition (HU>200 + 3D CC) — distribution-independent fallback. L2) Ds539 argmax masks + the V1.3 multi-anatomy routing above. L3) post-pad bbox sanity (≤ 50 % of volume) — rejects OOD masks. L4) 480 s time budget — guarantees the 10-minute GC limit. Largest-CC-keep per anatomy.

7. Self-diagnostic logging [V1.3.1; w0sum diagnosis confirmed in V1.3.2]

The container logs, every run: the **loaded network class** (STU-Net vs the plans' ResEnc) plus a **weight checksum** `w0sum` (the abs-sum of the stem conv); the **input raw-HU distribution** (min/max/mean/percentiles, bone>200 and air<-500 fractions); and **each Ds539 anatomy's volume fraction with a GARBAGE flag**. This logging is what pinned the V1.3.2 root cause: `w0sum` was **different on every GC case** ($\approx 80.5, 81.7$) instead of the trained checkpoint's constant **104.03** — the signature of an *unloaded, randomly-initialised* network (see the header). The fix restores a constant `w0sum=104.03` (Ds539) / `194.38` (Ds538) on the GC container, so a 0-score run is self-explaining from the log.

8. Performance (held-out, dev proxy)

Held-out grouped fold-0 val (n=132 per-anatomy ROIs), scored with our PENGWIN-2026 **official-aligned v2 proxy** (per-anatomy argmax IoU ≥ 0.10):

metric	overall	Femur	RightHip	LeftHip	Sacrum
Fracture Dice	0.799	0.845	0.854	0.759	0.732
Instance F1	0.919	0.953	0.950	0.876	0.892
HD95 (mm)	18.9	7.4	25.7	25.7	18.3

Scope of this proxy. It scores **Stage 2 on GT-derived ROIs** (oracle Stage-1 anatomy), so it measures the fracture decoder, not the full cascade. True end-to-end additionally depends on Stage-1 anatomy + the V1.3 routing (section 3); an end-to-end batch over full cases reaches mean fracture Dice ~ 0.97 when routed correctly. The official Grand-Challenge evaluator is unpublished; these are aligned proxies, not leaderboard results.

9. Hardware + runtime

NVIDIA T4 16 GiB; STU-Net-B inference ~ 4 GiB/stage (serial load, peak = max of the two stages); per-case 45-205 s, capped at 480 s. Container: PyTorch 2.1.2 + CUDA 11.8 + nnUNetv2 2.5.1.

10. Limitations

- **Dev measurement only** — the official test set is unreleased.

- The held-out proxy is Stage-2-on-oracle-ROIs (section 8); end-to-end is gated by Stage-1 anatomy + routing.
- Single fold, single seed; no ensembling.

11. References

- nnU-Net v2: Isensee, F. et al. *Nat Methods* 18, 203-211 (2021).
- STU-Net (TotalSegmentator bone pretrain).
- PENGWIN 2026 Task 1 baseline:
github.com/YzzLiu/PENGWIN2026_Task1_AutoSeg_Baseline
- PENGWIN 2024 1st place (ABBC reference): MIC-DKFZ.